

First program ChatGPT

- Name of two GenAI programs: **ChatGPT, Claude code**
- How you accessed them (e.g., Google query, CoPilot on my computer): **My openAI account web interface and anthropic account web interface**
- Prompt used. (You must use the same prompt for both GenAIs.)

prompt used:

can you make a C program that has the following requirements Objective Write a C program that picks a secret number between 1 and 10. The user gets up to 3 tries to guess the number. Requirements • Use a fixed secret number (for example, 7). • Use printf to prompt the user and scanf to read their guess. • Use a while or for loop to let the user guess up to 3 times. • Use if-else statements to check if the guess is too high, too low, or correct. • Stop the loop immediately if the user guesses the correct number. • Print a final win or lose message. Example Output text Guess a number between 1 and 10. Attempt 1/3. Enter your guess: 5 Too low! Try again. Attempt 2/3. Enter your guess: 8 Too high! Try again. Attempt 3/3. Enter your guess: 7 Correct! You win!

- Generated code from both GenAIs.

Chat GPT

```
#include <stdio.h> // includes input/output functions

int main() { // starts the main function

    int secretNumber = 7; // sets the secret number

    int guess; // stores the user's guess

    int attempt = 1; // tracks the current attempt

    int won = 0; // tracks whether the user won

    printf("Guess a number between 1 and 10.\n"); // prompts the user

    while (attempt <= 3) { // allows up to 3 attempts

        printf("Attempt %d/3.\n", attempt); // shows the attempt number

        printf("Enter your guess: "); // asks for a guess

        scanf("%d", &guess); // reads the user's guess

        if (guess == secretNumber) { // checks if the guess is correct

            printf("Correct! You win!\n"); // tells the user they won

            won = 1; // records that the user won

            break; // stops the loop

        } else if (guess < secretNumber) { // checks if the guess is too low

            printf("Too low! Try again.\n"); // tells the user

        } else { // runs if the guess is too high

            printf("Too high! Try again.\n"); // tells the user

        }

    }

}
```

```

        attempt++; // next attempt
    }

    if (won == 0) { // checks if the user lost

        printf("You lose! The secret number was %d.\n", secretNumber); // shows the lose message
    }

    return 0; // ends the program
} // kill the main function

```

Claude

```

#include <stdio.h> // include standard I/O library for printf and scanf

int main() { // start of main function

    int secret = 7; // fixed secret number the user must guess

    int guess; // variable to store the user's guess each attempt

    int max_tries = 3; //most guesses allowed

    int won = 0; // flag to track if the user has won (0 = not yet, 1 = won)

    printf("Guess a number between 1 and 10.\n"); // print initial instructions to the user

    for (int attempt = 1; attempt <= max_tries; attempt++) { // loop from attempt 1 up to max_tries

        printf("Attempt %d/%d. Enter your guess: ", attempt, max_tries); // show current attempt number and prompt for input

        scanf("%d", &guess); // read the user's guess into the guess variable

        if (guess < secret) { // check if the guess is lower than the secret number

            printf("Too low! Try again.\n"); // tell the user their guess was too low

        } else if (guess > secret) { // check if the guess is higher than the secret number

            printf("Too high! Try again.\n"); // tell the user their guess was too high

        } else { // the guess must be correct

            printf("Correct! You win!\n"); // tell the user they are guessing correctly

            won = 1; // set the won flag to 1 to indicate success

            break; // exit the loop immediately since the guess was correct

        }

    }

    if (!won) { // check if the user never guessed correctly

        printf("You lose! The number was %d.\n", secret); // reveal the secret number and tell the user they lost

    }

    return 0; // return 0 to indicate the program ended successfully
}

```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Correctness:

Claude:

Good: the program runs and works correctly. It follows all the requirements of the program and uses a for loop and runs until there are 3 guesses or correct answer.

Bad: the scanf function will work if the user inputs a numeric input but if a non-numeric is entered then it fails to assign a value this will cause the program to break and have undefined behavior. The bounds of 1-10 are never enforced so a user could put any value

Chatgpt:

Good: the program runs and works correctly. It follows all the requirements of the program and uses a while loop and runs until there are 3 guesses or correct answer.

Bad: same scanf issue as Claude's code

Execution time:

Claude:

Good: uses a for loop which runs faster and is usually more hard coded than a while loop which it could have chosen.

Bad: no issues

Chatgpt:

Good: while it uses a while loop this will still run very fast with no issues of speed

Bad: uses a while loop, for this example this is negligible but for other examples this might become impactful

Space complexity:

Claude:

Good: memory storage is small and minimal the program uses only the number of variables it needs

Bad: no issues

Chatgpt:

Good: memory storage is small and minimal the program uses only the number of variables it needs

Bad: no issues

Maintainability:

Claude:

Good: variable names are clear and show functions.

Bad: the secret number is hard coded thus it might be better to have the standard all caps SECRET_NUMBER to show it is a constant

Chatgpt:

Good: variable names are clear and show functions.

Bad: the secret number is hard coded thus it might be better to have the standard all caps SECRET_NUMBER to show it is a constant. In this version 3 is hard coded for attempts this would be an issue if someone wanted to go to the code and change the number of attempts this makes it hard to find. No 1-10 range finding.

f. Select one to use as the basis for this assignment and justify why you picked it.

I chose the Claude code example this one will run faster and less error prone than the while loop implemented in the GPT version. Also Claude is more maintainable with the attempts variable.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness: added the valid variable so that the program only accepts a number between 1-10 as the rules stipulate, it then can handle the error.

Execution time: none

Space complexity: none

Maintainability: I made Secret_number to SECRET_NUMBER so it is clearly a constant, and MAX_TRIES to also be a constant.

3. Make the changes identified in 2g above.

Made changes to the Claude code.

4. Compile and debug your program as described in the "C Basics" lecture.

5. Push both your source code, final cycle server executable file, and PDF of your

GenAI analysis to your GitHub repository. https://github.com/CarterGary-17/Fall2026_EECS348_Assignment_1